

BEGINNER LEARNING GUIDE

System Design for Beginners

300 Questions and Answers — From Beginner to Advanced

A practical, beginner-friendly guide to the core concepts
behind scalable, reliable software systems.

Created By: Kartik Mungole · @kartikmungole

Q001 What is system design?

System design is the process of planning how a software application will work before you start building it. It means deciding how the different parts, like the website, the database, and the servers, will fit together and talk to each other. Think of it like an architect drawing blueprints for a house before construction begins, instead of just starting to lay bricks randomly.

Q002 Why is system design important?

System design is important because it helps engineers build applications that can handle real users and real problems, not just work perfectly in a small test. Without good design, an app might work fine for 10 users but completely crash when 10,000 people try to use it at once. Planning ahead helps avoid expensive rebuilding later, saving both time and money.

Q003 What is the difference between system design and coding?

Coding is the act of writing the actual instructions a computer follows, line by line, in a programming language. System design is the bigger picture plan that comes before coding, deciding how different pieces of code and services should be organized and connected. You can think of coding as building individual rooms, while system design is the overall floor plan of the entire house.

Q004 What is a system in software engineering?

A system is a group of components that work together to perform a specific task, such as running a website, processing payments, or sending messages. For example, an online shopping system includes a website for browsing, a database to store products, a payment service, and a way to track orders. Each piece does its own job, but together they form one complete system.

Q005 What is high-level design?

High-level design is the overall plan for a system, showing the main components and how they connect, without going into small details. For example, a high-level design for a chat app might just show boxes labeled 'app,' 'server,' and 'database,' with arrows showing how messages flow between them. It is meant to give a quick, easy-to-understand overview before diving into specifics.

Q006 What is low-level design?

Low-level design is the detailed plan for how each individual part of a system actually works on the inside. While high-level design might just show a box labeled 'database,' low-level design would explain exactly which tables exist, what columns they have, and how the code reads and writes to them. It is the next step after highlevel design, filling in all the specific details needed to actually build the system.

Q007 Who uses system design skills?

System design skills are used by software engineers, technical architects, and team leads who need to plan applications that can grow and stay reliable over time. Even beginner programmers benefit from learning system design, because it helps them write code that fits well into a larger application instead of code that only works in isolation. Many companies also test system design knowledge in job interviews, especially for more experienced engineering roles.

Q008 What is a client in system design?

A client is the application or device that a person directly uses and interacts with, such as a web browser, a mobile app, or a desktop program. When you open Instagram on your phone, your phone is acting as the client. The client's job is to display information to the user and send requests when the user wants something, like loading a new page or sending a message.

Q009 What is a server in system design?

A server is a powerful computer or program that waits for requests from clients, processes them, and sends back a response. For example, when you search something on Google, your browser, the client, sends a request to Google's servers, which then search for results and send them back to your screen. Servers are usually always running and ready to handle many requests at once.

Q010 What is a client-server architecture?

A client-server architecture is a design where many clients send requests to one central server, and that server processes those requests and sends back responses. This is one of the most common patterns in software, used by almost every website and app you use daily. The benefit is that the server can manage and control everything in one place, while many different clients can connect to it from anywhere.

Q011 What is a request and a response?

A request is a message a client sends to a server asking for something, like 'show me this webpage' or 'save this new comment.' A response is the message the server sends back, which could be the requested webpage, a confirmation message, or an error if something went wrong. This back-and-forth communication happens constantly, often within a fraction of a second, every time you use an app or website.

Q012 What is an IP address?

An IP address is a unique number assigned to every device connected to a network, similar to how every house has a unique street address. Computers use IP addresses to know exactly where to send data on the internet, since there are billions of connected devices. For example, when you visit a website, your computer is really sending a request to that website's specific IP address behind the scenes.

Q013 What is a domain name?

A domain name is the easy-to-read website address you type into your browser, like google.com or amazon.com, instead of a long string of numbers. It exists because IP addresses are hard for humans to remember, so domain names act as friendly nicknames for them. When you type a domain name, your computer quietly looks up the matching IP address before connecting to the right server.

Q014 What is DNS?

DNS, which stands for Domain Name System, is the service that translates domain names into IP addresses behind the scenes. It works like a giant phonebook for the internet: you give it a name, like example.com, and it tells your computer the actual numeric address to connect to. Without DNS, you would have to memorize long strings of numbers instead of simple website names.

Q015 What is a database?

A database is an organized system for storing data so it can be easily saved, searched, and updated later. Think of it like a digital filing cabinet that an application uses to remember information, such as user accounts, orders, or messages, even after the app is closed and reopened. Without a database, an app would forget everything every time it restarted.

Q016 What is a relational database?

A relational database stores data in tables made up of rows and columns, very similar to a spreadsheet. For example, a 'Users' table might have columns for name, email, and age, with each row representing one person. Multiple tables can be linked together, such as connecting a 'Users' table to an 'Orders' table, so you can see which user placed which order.

Q017 What is a non-relational database?

A non-relational database, often called NoSQL, stores data in more flexible formats instead of strict tables with fixed columns. For example, it might store each user's information as a flexible document that can have different fields for different users. This flexibility makes non-relational databases useful for data that does not fit neatly into rows and columns, like chat messages or product catalogs with varying details.

Q018 What is SQL?

SQL, which stands for Structured Query Language, is the language used to communicate with relational databases. With SQL, you can write commands to add new data, search for existing data, update information, or delete records you no longer need. For example, a simple SQL command might ask the database to 'find all users older than 18,' and the database will return exactly that list.

Q019 What is a primary key in a database?

A primary key is a special column in a database table that uniquely identifies each row, ensuring no two rows are ever confused with each other. For example, in a 'Users' table, each user might have a unique ID number as their primary key, even if two users happen to share the same name. This makes it easy and reliable for the system to find one exact record whenever it needs to.

Q020 What is a foreign key in a database?

A foreign key is a column in one table that points to the primary key of another table, creating a connection between the two. For example, an 'Orders' table might have a 'user_id' column that matches the primary key in the 'Users' table, showing which user placed each order. This linking is how relational databases keep related pieces of information connected without duplicating data everywhere.

Q021 What is scalability?

Scalability is the ability of a system to keep working well even as more people use it or more data is added. A scalable app might work smoothly whether it has 100 users or 1 million users, while a poorly designed app might slow down or crash as it grows. Planning for scalability early on helps avoid painful rebuilds later when an app suddenly becomes popular.

Q022 What is vertical scaling?

Vertical scaling means making a single server stronger by giving it more memory, a faster processor, or more storage, so it can handle more work on its own. It is similar to upgrading one employee to work faster instead of hiring more people. The downside is that there is always a limit to how powerful a single machine can become, no matter how much money you spend.

Q023 What is horizontal scaling?

Horizontal scaling means adding more servers to share the workload, instead of making one server more powerful. It is similar to hiring more employees to handle increasing work, rather than asking one employee to work impossibly fast. This approach is usually preferred for very large systems because you can keep adding more servers almost indefinitely as your needs grow.

Q024 What is the difference between vertical and horizontal scaling?

Vertical scaling upgrades a single existing machine, while horizontal scaling adds more machines to share the work. Vertical scaling is often simpler at first but eventually hits a hard limit, since no single computer can be infinitely powerful. Horizontal scaling takes more setup but can grow much larger over time, which is why most big websites rely heavily on it.

Q025 What is a bottleneck in system design?

A bottleneck is the slowest part of a system that limits how fast the entire system can perform, no matter how fast the other parts are. Imagine a highway that suddenly narrows from four lanes to one lane; even if the rest of the highway is wide and fast, traffic will still back up at that narrow point. In software, a bottleneck might be a slow database query that delays every single request, even if everything else in the system is lightning fast.

Q026 What does it mean for a system to be highly available?

A highly available system is designed to stay up and running as much as possible, with very little unexpected downtime. This often involves having backup servers ready to take over instantly if one server fails, so users barely notice anything went wrong. Companies like banks and online stores invest heavily in high availability because even a few minutes of downtime can mean lost money and frustrated customers.

Q027 What is a load balancer?

A load balancer is a component that sits in front of multiple servers and spreads incoming requests evenly across them, instead of sending everything to just one server. Imagine a restaurant host directing customers to different open tables instead of letting everyone crowd around a single table while others sit empty. This keeps any one server from becoming overwhelmed while other servers sit idle and unused.

Q028 Why is load balancing important?

Load balancing is important because it helps a system stay fast and reliable, even when traffic suddenly spikes, such as during a big sale or viral event. Without it, one server might crash from too many requests while other available servers are barely being used at all. Spreading the work evenly means the whole system can handle much more traffic smoothly, without any single point becoming a problem.

Q029 What is caching?

Caching is the practice of storing a copy of frequently used information somewhere fast to access, so the system does not have to redo the same slow work over and over again. For example, instead of recalculating the same popular search result every single time someone asks for it, the system can save, or cache, that result and reuse it instantly for the next person. This saves time and reduces unnecessary work for the rest of the system.

Q030 Why is caching useful in system design?

Caching is useful because it dramatically speeds up response times and reduces the load placed on databases and servers. Imagine if every time you opened a popular webpage, the server had to redo a complicated calculation from scratch; that would be incredibly slow for everyone. By caching the result once and reusing it, thousands of users can get fast responses without repeatedly straining the system.

Q031 What is a CDN?

A CDN, or Content Delivery Network, is a group of servers placed in many different locations around the world that store copies of website content, like images and videos. When a user requests content, the CDN serves it from the server closest to that user's actual location, instead of one faraway central server. This means someone in Japan and someone in Brazil can both get fast loading times, because they are each served from a nearby copy.

Q032 What is cache invalidation?

Cache invalidation is the process of removing or refreshing old cached data once the original, real data changes. For example, if a product's price changes in the database but the old price is still cached, customers might see the wrong price until the cache is updated. Properly handling cache invalidation is tricky but important, because showing outdated information can confuse or mislead users.

Q033 What is an API?

An API, or Application Programming Interface, is a defined way for different software applications to talk to each other and exchange information. You can think of it like a restaurant menu: it lists exactly what you can ask for, without needing to know how the kitchen actually prepares the food. For example, a weather app might use an API to ask a weather service for today's forecast, without needing to know how that service collects its data.

Q034 What is a REST API?

A REST API is one of the most common types of APIs, using standard web requests like GET, to read data, and POST, to create new data, so different applications can communicate easily over the internet. For example, a mobile shopping app might use a REST API to fetch the list of products from a server and display them to the user. REST APIs are popular because they are simple, predictable, and widely supported across almost every programming language.

Q035 What is HTTP?

HTTP, which stands for HyperText Transfer Protocol, is the basic set of rules computers follow to send and receive information over the internet, especially webpages. Every time you load a website, your browser is using HTTP behind the scenes to ask a server for that page's content. It defines things like how a request should be formatted and what kind of response the server should send back.

Q036 What is the difference between synchronous and asynchronous communication?

Synchronous communication means the sender sends a request and then waits, doing nothing else, until a response comes back. Asynchronous communication means the sender can continue doing other work right away, and the response arrives later without blocking everything else. A simple analogy is making a phone call, which is synchronous because you wait on the line, versus sending a text message, which is asynchronous because you can do other things while waiting for a reply.

Q037 What is a message queue?

A message queue is a system that temporarily holds messages sent between different parts of an application, allowing them to communicate without needing an instant response. For example, when you upload a video, the upload service might place a message in a queue saying 'please process this video,' and a separate service picks it up and processes it whenever it is ready. This means the upload finishes quickly for the user, even though the actual video processing might take longer in the background.

Q038 Why are message queues useful in system design?

Message queues are useful because they let different parts of a system work independently, at their own speed, without forcing everything to happen instantly and in perfect sync. This makes systems more resilient, since if one service is temporarily slow or down, messages can simply wait safely in the queue until that service is ready again. It also helps spread out heavy workloads over time instead of overwhelming a system all at once.

Q039 What is reliability in system design?

Reliability is the ability of a system to keep working correctly over a long period of time, even when unexpected problems happen, such as hardware failures, bugs, or sudden traffic spikes. A reliable system behaves consistently and predictably, so users can trust it to work when they need it. Building reliability often means planning for things to go wrong and having a plan ready, instead of simply hoping nothing breaks.

Q040 What is fault tolerance?

Fault tolerance is the ability of a system to keep functioning, possibly with slightly reduced performance, even when one or more parts of it fail. For example, if one server in a group of ten crashes, a fault-tolerant system simply continues running on the remaining nine servers without users even noticing. This is different from a fragile system, where a single failure could bring the entire application crashing down.

Q041 What is redundancy in system design?

Redundancy means having backup copies of important components, such as extra servers, extra databases, or extra power supplies, so the system can keep running even if one part fails. It is similar to a building having a backup generator in case the main power goes out. While redundancy costs extra money and resources, it provides valuable protection against unexpected failures.

Q042 What is a single point of failure?

A single point of failure is any part of a system that, if it stops working, causes the entire system to fail, because nothing else can take over its job. For example, if an entire website relies on just one server with no backup, that one server is a single point of failure. Good system design tries to identify and remove these weak points by adding backups or alternative paths wherever possible.

Q043 What is a failover?

A failover is the automatic process of switching over to a backup system or server the moment the main one fails, so the overall application keeps running smoothly. For example, if a primary database suddenly goes offline, a failover system might automatically switch all traffic to a backup database within seconds. This automatic switching is designed to happen quickly enough that most users never even realize something went wrong.

Q044 What is downtime?

Downtime is any period of time during which a system is not working properly or is completely unavailable to its users. For example, if a website cannot be reached for 10 minutes due to a server crash, that 10 minute period counts as downtime. Companies often track downtime closely, since too much of it can mean lost revenue, frustrated customers, and damaged trust in the product.

Q045 What does a simple web application's system design usually include?

A simple web application's system design usually includes a client, like a web browser, a server to process requests, and a database to store information such as user accounts or posts. As an app grows, it might also add a load balancer to spread traffic across multiple servers and a cache to speed up frequently requested data. Even very large, complex systems are often just bigger combinations of these same basic building blocks.

Q046 What is the role of monitoring in system design?

Monitoring is the ongoing practice of tracking how a system is performing in real time, such as how fast it responds and whether any errors are happening. It acts like a dashboard of warning lights, alerting engineers quickly when something starts going wrong, often before users even notice a problem. Without monitoring, small issues can quietly grow into major outages before anyone realizes there is a problem.

Q047 What is capacity planning?

Capacity planning is the process of estimating how much traffic, storage, or processing power a system will likely need in the future, based on expected growth. For example, an online store might plan extra server capacity ahead of a big holiday sale, knowing traffic will spike heavily on that day. Good capacity planning helps avoid systems being caught off guard and crashing exactly when they are needed the most.

Q048 What is the difference between latency and throughput?

Latency is how long it takes to get a single response back after making one request, almost like asking one question and waiting for one answer. Throughput is how many requests or tasks a system can handle within a given period of time, like how many questions can be answered per minute overall. A system can technically have low latency, meaning each individual response is fast, but still have limited throughput if it can only handle a small number of requests at once.

Q049 What is a trade-off in system design?

A trade-off happens when improving one part of a system means giving up or weakening something else, since you usually cannot maximize everything at the same time. For example, making a system extremely fast might require expensive extra hardware, while keeping costs low might mean accepting slightly slower performance. Recognizing and consciously choosing these trade-offs, instead of ignoring them, is one of the most important skills in good system design.

Q050 What is the key takeaway for beginners learning system design?

The key takeaway for beginners is to start by deeply understanding the basic building blocks, such as clients, servers, databases, caching, and load balancing, since almost every system is built from combinations of these same simple ideas. Focus on understanding why each piece exists and what real-world problem it solves, rather than just memorizing definitions. Once these fundamentals feel comfortable, more advanced topics like large-scale distributed systems will become much easier to learn, because they are really just bigger versions of the same basic concepts. Key Takeaway: System design is about understanding how the basic pieces of an application work together. Master clients, servers, databases, scalability, and reliability, and you will have a strong foundation for any system design discussion.

Q051 What is HTTP?

HTTP, or HyperText Transfer Protocol, is the set of rules that browsers and servers use to talk to each other over the internet. When you type a web address, your browser sends an HTTP request asking for a page, and the server sends back an HTTP response containing that page. It is the foundation that almost every website and web API is built on top of.

Q052 What is the difference between HTTP and HTTPS?

HTTPS is simply HTTP with an added layer of encryption called TLS, which scrambles the data so no one snooping on the network can read it. Without HTTPS, information like passwords or credit card numbers could be intercepted in plain text as it travels between the browser and server. Nearly every modern website uses HTTPS by default to keep user data private and secure.

Q053 What are common HTTP methods?

The most common HTTP methods are GET for reading data, POST for creating new data, PUT and PATCH for updating existing data, and DELETE for removing data. Each method signals the intent of the request, which helps both developers and servers understand what action should happen. Using the right method consistently makes APIs easier to predict and work with.

Q054 What are HTTP status codes?

HTTP status codes are three digit numbers that a server sends back to tell the client what happened with a request, such as 200 for success or 404 for not found. Codes starting with 2 mean success, 4 mean the client made a mistake, and 5 mean the server had a problem. Recognizing these codes quickly helps developers debug issues without digging through logs.

Q055 What is DNS?

DNS, or Domain Name System, is like the phonebook of the internet that translates human-friendly website names into numerical IP addresses computers use to find each other. When you type a domain into your browser, DNS looks up the matching IP address so your request can be sent to the right server. Without DNS, everyone would need to memorize long strings of numbers instead of simple names.

Q056 How does a DNS lookup work?

A DNS lookup usually starts at your browser and device, then checks a local cache before asking a DNS resolver, which may query root, TLD, and authoritative name servers if needed. Each server in the chain knows a little more about where to find the final answer, similar to asking for directions and getting pointed closer each time. The whole process typically takes just milliseconds thanks to caching along the way.

Q057 What is DNS caching?

DNS caching means storing the result of a domain lookup temporarily so future requests for the same domain do not need to repeat the entire lookup process. This caching can happen at the browser, the operating system, or intermediate DNS servers, and it greatly speeds up how quickly websites load. Each cached entry has a time to live, after which it expires and must be looked up again.

Q058 What is a CNAME record?

A CNAME record is a type of DNS entry that points one domain name to another domain name instead of directly to an IP address. This is useful when you want a subdomain like `blog.example.com` to simply follow whatever address `example.com` currently points to. It makes managing multiple related domains easier since you only need to update one underlying record.

Q059 What is a REST API?

A REST API is a way for applications to communicate over HTTP using a consistent set of conventions built around resources, such as users or orders, identified by URLs. Each resource can be created, read, updated, or deleted using standard HTTP methods, which makes the API predictable and easy to learn. REST has become the most common style for building web APIs because of its simplicity.

Q060 What does it mean for an API to be stateless?

A stateless API means the server does not remember anything about previous requests from a client; every request must contain all the information needed to process it. This makes it much easier to scale a system, since any server in a pool can handle any request without needing shared memory of past interactions. Statelessness is one of the core principles behind REST design.

Q061 What is a resource in REST?

A resource in REST is any piece of data or object the API exposes, such as a specific user, product, or blog post, usually represented by a unique URL. For example, `/users/42` might represent the user with ID 42, and different HTTP methods on that URL let you view, update, or delete that particular user. Thinking in terms of resources helps keep API design clean and consistent.

Q062 What is API versioning and why is it needed?

API versioning is the practice of labeling different versions of an API, often with something like `/v1/` or `/v2/` in the URL, so changes do not break existing users. It allows a team to improve or redesign an API while still supporting older applications that have not updated yet. Without versioning, a single change could unexpectedly break every app currently depending on that API.

Q063 What is GraphQL?

GraphQL is a query language for APIs that lets clients ask for exactly the data they need in a single request, instead of receiving a fixed structure from the server. Unlike REST, where you might need to call several endpoints to gather related data, GraphQL lets you request nested and related fields all at once. It was created by Facebook to make mobile apps faster by reducing unnecessary data transfer.

Q064 How is GraphQL different from REST?

REST typically exposes many fixed endpoints, each returning a predetermined shape of data, while GraphQL exposes a single endpoint where clients describe exactly what fields they want. This flexibility in GraphQL can reduce the number of network calls and avoid over-fetching unused data. However, REST is often simpler to cache and reason about, since its endpoints and responses are more predictable.

Q065 What is over-fetching and under-fetching?

Over-fetching happens when an API response includes more data than the client actually needs, wasting bandwidth and processing time. Under-fetching happens when a single API call does not return enough data, forcing the client to make several follow-up requests to gather everything it needs. GraphQL was designed specifically to solve both of these problems by letting clients request precisely the fields they want.

Q066 What is a GraphQL schema?

A GraphQL schema is a strongly typed definition of all the data types, fields, and relationships that a GraphQL API supports, acting like a contract between client and server. It tells developers exactly what queries are possible and what shape the results will take, which makes the API self-documenting. Tools can automatically generate documentation and even test queries directly from this schema.

Q067 What is gRPC?

gRPC is a high-performance framework for making remote procedure calls between services, allowing one program to call a function on another program as if it were local. It was built by Google and is commonly used for fast communication between internal microservices rather than public-facing APIs. gRPC uses HTTP/2 under the hood, which allows multiple requests to be sent efficiently over a single connection.

Q068 What are Protocol Buffers?

Protocol Buffers, often called protobuf, are a compact binary format used to define and serialize structured data, commonly paired with gRPC. Instead of sending verbose text like JSON, protobuf converts data into a smaller binary form that is faster to transmit and parse. Developers define the data structure once in a .proto file, and code can be generated automatically for many programming languages.

Q069 Why is gRPC often faster than REST with JSON?

gRPC is often faster than REST with JSON because it uses compact binary encoding instead of readable text, which reduces the amount of data sent over the network. It also uses HTTP/2, which supports multiplexing multiple requests over one connection instead of opening a new connection for each call. These combined improvements make gRPC a popular choice for internal service-to-service communication where speed really matters.

Q070 When would you choose gRPC over REST?

You would typically choose gRPC over REST when building internal microservices that need to communicate quickly and efficiently with strict data contracts. REST tends to be a better fit for public APIs where broad compatibility and human readability, like browsers easily reading JSON, matter more than raw performance. Many companies actually use both, gRPC internally and REST for external-facing endpoints.

Q071 What is a WebSocket?

A WebSocket is a communication protocol that keeps a single connection open between a client and server, allowing both sides to send messages to each other at any time. This differs from regular HTTP requests, which close the connection right after each response. WebSockets are ideal for applications like chat apps or live dashboards where data needs to update instantly without repeated requests.

Q072 How do WebSockets differ from regular HTTP requests?

Regular HTTP requests follow a request-response pattern where the client always initiates communication and the connection closes afterward.

WebSockets instead establish a persistent, two-way connection, so either the client or the server can send data at any moment without waiting to be asked. This makes WebSockets much more efficient for real-time features than repeatedly opening new HTTP connections.

Q073 What is a use case where WebSockets are essential?

A live chat application is a classic use case for WebSockets, since messages need to appear instantly for all participants without anyone having to refresh the page. Online multiplayer games and real-time stock tickers are other strong examples, since both require constant, immediate updates flowing in both directions. In all these cases, the always-open connection avoids the delay of setting up a new request every time.

Q074 What is the tradeoff of using WebSockets at scale?

WebSockets require the server to keep an open connection for every single connected client, which uses far more memory and resources than stateless HTTP requests. This means a system supporting millions of WebSocket connections needs careful capacity planning and often specialized infrastructure to manage all those persistent connections. Because of this cost, WebSockets are usually reserved for features that genuinely need real-time updates.

Q075 What is polling in system design?

Polling is when a client repeatedly asks a server, at regular intervals, whether there is new information available, even if nothing has actually changed. It is simple to implement, but wastes resources when most of those requests come back with no new data. Polling frequency is often a tradeoff between how quickly updates are noticed and how much unnecessary load is placed on the server.

Q076 What is a webhook?

A webhook is a way for a server to notify another system automatically the moment something happens, instead of that system having to repeatedly ask if anything changed. For example, a payment provider might send a webhook to your server the instant a payment succeeds, rather than your server constantly checking. Webhooks are essentially the server pushing information out, flipping the usual request direction.

Q077 How do webhooks reduce unnecessary network traffic?

Webhooks reduce unnecessary network traffic because the receiving system only gets a message when there is actually something new to report, unlike polling which sends requests regardless of whether anything changed. This means far fewer wasted calls, especially for events that happen rarely, like a rare account status change. The tradeoff is that the receiving system must expose an endpoint that is always ready to accept incoming webhook calls.

Q078 What is long polling?

Long polling is a middle ground between regular polling and WebSockets, where the client sends a request and the server holds it open until new data is available or a timeout occurs. Once the server responds, the client immediately sends another request to keep waiting for the next update. It is more efficient than constant short polling, though generally less efficient than a persistent WebSocket connection.

Q079 What is TCP?

TCP, or Transmission Control Protocol, is a reliable way of sending data across a network that guarantees packets arrive in order and none are lost along the way. It achieves this by acknowledging received packets and resending anything that gets dropped, though this reliability adds some extra delay. Most web traffic, including HTTP, relies on TCP because correctness matters more than raw speed.

Q080 What is UDP?

UDP, or User Datagram Protocol, sends data quickly without guaranteeing that packets arrive, arrive in order, or arrive at all, trading reliability for speed. This makes UDP well suited for applications like video calls or online gaming, where a few lost packets are less harmful than noticeable delay. Unlike TCP, there is no handshake or acknowledgment system slowing things down.

Q081 When would you choose UDP over TCP?

You would choose UDP over TCP when speed and low latency matter more than perfect reliability, such as live video streaming, voice calls, or fast-paced online games. In these cases, a slightly corrupted or missing frame is far less noticeable than the lag caused by waiting for retransmitted data. TCP remains the better choice whenever complete accuracy, like loading a webpage or transferring a file, is essential.

Q082 What is a three-way handshake?

A three-way handshake is the process TCP uses to establish a reliable connection before any real data is sent, involving three steps: SYN, SYN-ACK, and ACK. The client first says it wants to connect, the server acknowledges and agrees, and then the client confirms back, after which data can flow safely. This handshake ensures both sides are ready and helps prevent errors before real communication starts.

Q083 What is an IP address?

An IP address is a unique numerical label assigned to every device connected to a network, allowing other devices to find and communicate with it. It works similarly to a street address, letting data packets know exactly where to be delivered across the internet. There are two common formats in use today, called IPv4 and the newer, larger IPv6.

Q084 What is the difference between a public and private IP address?

A public IP address is visible on the open internet and uniquely identifies a device or network to the outside world, like your home router's internet-facing address. A private IP address is only used within a local network, such as your home Wi-Fi, and is not directly reachable from the internet. Many devices behind a single router share one public IP while each having their own private IP internally.

Q085 What is a port number?

A port number is a number attached to network communication that identifies which specific application or service on a device should receive the data. For example, web traffic commonly uses port 80 for HTTP and port 443 for HTTPS, while a database might listen on a completely different port. Ports let a single device run many different network services at the same time without conflicts.

Q086 What is NAT and why is it used?

NAT, or Network Address Translation, allows many devices on a private network to share a single public IP address when communicating with the internet. It works by rewriting the source address of outgoing packets and keeping track of which internal device should receive the response. NAT helped delay the exhaustion of IPv4 addresses by letting entire households or offices share just one public address.

Q087 What is a proxy server?

A proxy server sits between a client and the internet, forwarding requests on the client's behalf, which can help with privacy, filtering, or caching. For example, a company might route all employee internet traffic through a proxy to block certain websites or monitor usage. From the destination server's perspective, the request appears to come from the proxy rather than the original client.

Q088 What is a reverse proxy?

A reverse proxy sits in front of one or more backend servers and forwards client requests to the appropriate server behind it, hiding the backend's details from the outside world. It is commonly used for load balancing, SSL termination, and adding an extra layer of security in front of application servers. Popular examples of reverse proxy software include Nginx and HAProxy.

Q089 What is the difference between a proxy and a reverse proxy?

A regular proxy works on behalf of the client, hiding the client's identity from the servers it talks to, while a reverse proxy works on behalf of the server, hiding the server's identity from clients. Both sit in the middle of a connection, but the direction of who they protect is reversed. Many production systems use reverse proxies to manage incoming traffic before it ever reaches application servers.

Q090 What are common benefits of using a reverse proxy?

A reverse proxy can distribute incoming traffic across multiple servers, handle SSL certificate management in one central place, and cache frequently requested content to reduce backend load. It can also add security by hiding internal server details and blocking suspicious traffic before it reaches the actual application. These combined benefits make reverse proxies a standard part of most production web architectures.

Q091 What is an API gateway?

An API gateway is a single entry point that sits in front of a system's backend services, handling incoming requests before routing them to the right place. It can manage tasks like authentication, rate limiting, and logging in one central location instead of repeating that logic in every individual service. This simplifies client interactions since they only need to know about one address rather than dozens of internal services.

Q092 Why are API gateways useful in a microservices architecture?

In a microservices architecture, an API gateway prevents clients from needing to know about or directly call dozens of separate internal services. It consolidates cross-cutting concerns like authentication, logging, and rate limiting into a single layer instead of duplicating that logic across every service. This makes the overall system easier to secure, monitor, and evolve over time.

Q093 What responsibilities can an API gateway handle?

An API gateway can handle request routing, authentication and authorization checks, rate limiting, request and response transformation, and detailed logging of traffic. By centralizing these responsibilities, individual backend services can stay focused purely on their core business logic. This separation of concerns keeps the overall system cleaner and easier to maintain.

Q094 What is a downside of relying heavily on an API gateway?

An API gateway can become a single point of failure or a performance bottleneck if it is not designed with redundancy and scaling in mind. Since all traffic flows through it, any slowdown or outage at the gateway level can affect the entire system at once. Because of this, gateways are usually deployed with multiple redundant instances behind a load balancer.

Q095 What is rate limiting?

Rate limiting is a technique that restricts how many requests a client can make to a system within a given period of time, such as 100 requests per minute. It protects a system from being overwhelmed by too much traffic, whether from a sudden spike, a bug, or a malicious attack. Without rate limiting, a single misbehaving client could degrade performance for everyone else using the system.

Q096 What is the token bucket algorithm?

The token bucket algorithm is a popular way to implement rate limiting, where a bucket holds a limited number of tokens that refill at a steady rate over time. Each incoming request consumes one token, and if the bucket is empty, the request is delayed or rejected until more tokens become available. This approach allows short bursts of traffic while still enforcing an overall average limit.

Q097 Why is rate limiting important for public APIs?

Rate limiting is important for public APIs because it prevents any single user or application from consuming a disproportionate share of server resources. It also protects against abuse, such as bots scraping data aggressively or attackers attempting to overwhelm the system. Fair rate limits help ensure the API remains fast and reliable for all legitimate users.

Q098 What is the difference between rate limiting and throttling?

Rate limiting typically means rejecting or blocking requests once a set limit is exceeded within a time window, often returning an error to the client. Throttling usually means slowing down or queuing requests instead of rejecting them outright, smoothing out traffic spikes over time. Both techniques aim to protect system resources, but they respond to excess demand in slightly different ways.

Q099 What is backpressure in system design?

Backpressure is a signal that tells a sender to slow down because the receiver cannot keep up with the current rate of incoming data or requests. It prevents a fast producer from overwhelming a slower consumer, which could otherwise lead to crashes or dropped data. Systems that handle streaming data or message queues often need explicit backpressure mechanisms to stay stable.

Q100 How does throttling help protect a system?

Throttling helps protect a system by intentionally slowing down the rate at which requests are processed, rather than letting them all through at once and risking an overload. It smooths out sudden bursts of traffic into a more manageable, steady flow that the system can reliably handle. This trades a small amount of added delay for much greater overall stability.

Q101 What happens if a system does not implement backpressure?

Without backpressure, a fast producer can overwhelm a slower consumer, causing memory to fill up with unprocessed data, increased latency, or even complete crashes. Imagine a firehose pouring water into a cup that can only hold so much before it overflows. Systems without backpressure mechanisms often fail unpredictably under heavy load rather than degrading gracefully.

Q102 What is an example of backpressure in a real system?

A common example is a message queue that pauses or slows down a producer when the queue grows past a certain size, giving consumers time to catch up. Video streaming services also use backpressure by buffering data and adjusting quality when the network cannot keep up with the requested bitrate. In both cases, the system deliberately signals the source to reduce its rate rather than letting problems pile up silently.

Q103 What is a message queue?

A message queue is a system that temporarily stores messages sent from one part of an application until another part is ready to process them. Instead of one service calling another directly and waiting for an immediate response, it drops a message in the queue and moves on. This decouples the sender and receiver, so each can operate independently at their own pace.

Q104 Why are message queues useful in system design?

Message queues are useful because they let different parts of a system communicate without needing to be available or fast at exactly the same moment. If a receiving service is temporarily slow or down, messages simply wait safely in the queue instead of being lost. This improves reliability and allows systems to handle sudden bursts of activity more gracefully.

Q105 What is the difference between synchronous and asynchronous communication?

Synchronous communication means the sender waits for an immediate response before continuing, similar to a phone call where both people must be present at the same time. Asynchronous communication, often enabled by message queues, lets the sender continue working immediately while the message is processed later, similar to sending a text message. Asynchronous patterns are especially useful for tasks that do not need an instant result, like sending confirmation emails.

Q106 What is a dead letter queue?

A dead letter queue is a special queue where messages are placed if they repeatedly fail to be processed successfully after several attempts. Instead of endlessly retrying or silently losing the message, it gets set aside so engineers can inspect and debug what went wrong. This prevents one broken message from clogging up the main queue and blocking other messages from being processed.

Q107 What is the publish-subscribe pattern?

The publish-subscribe pattern, often called pub-sub, lets one part of a system announce, or publish, an event without knowing exactly who will receive it. Other parts of the system, called subscribers, express interest in certain types of events and automatically receive them when published. This decouples publishers from subscribers, so new subscribers can be added without changing the publisher at all.

Q108 How is pub-sub different from a simple message queue?

A simple message queue typically delivers each message to only one consumer, while pub-sub can broadcast a single message to many interested subscribers at once. Pub-sub is ideal when multiple different services all need to react to the same event, like a new order being placed. A traditional queue is better suited for distributing individual tasks among a pool of workers where each task should only be handled once.

Q109 What is a topic in a pub-sub system?

A topic is a named channel that publishers send messages to and that subscribers listen on to receive relevant messages. For example, a system might have a topic called order-created that various services, like billing and shipping, both subscribe to independently. Organizing messages by topic keeps related events grouped together and makes it easy to add new subscribers later.

Q110 What is a real-world example of pub-sub in action?

A real-world example is a social media platform notifying multiple services, such as the notification system, analytics system, and search index, whenever a user posts something new. Each of these services subscribes to the same post-created event and reacts independently without the posting service needing to know about them individually. This keeps the system flexible as new features and subscribers are added over time.

Q111 What is Apache Kafka?

Apache Kafka is a popular distributed streaming platform used to publish, store, and process large volumes of real-time data reliably. It is often used as the backbone for event-driven systems, handling everything from user activity tracking to log aggregation across many services. Kafka is designed to be highly scalable and fault tolerant, capable of processing millions of messages per second.

Q112 What is a Kafka topic and partition?

A Kafka topic is a named stream of messages, similar to a category, while a partition is a way of splitting that topic into smaller, ordered pieces for parallel processing. Splitting a topic into multiple partitions allows Kafka to spread the load across several servers, improving throughput significantly. Messages within a single partition are always kept in the order they were written.

Q113 What is a Kafka consumer group?

A Kafka consumer group is a set of consumers that work together to process messages from a topic, with each partition being handled by only one consumer in the group at a time. This allows multiple consumers to share the workload of processing a large topic in parallel while still avoiding duplicate processing within the group. Different consumer groups can independently read the same topic without interfering with each other.

Q114 Why is Kafka considered durable and reliable?

Kafka is considered durable because it writes messages to disk and can replicate them across multiple servers, so data survives even if one server fails. Consumers can also replay old messages by resetting their position, which is useful for reprocessing data or recovering from bugs. This combination of replication and persistent storage makes Kafka well suited for critical data pipelines.

Q115 What is event-driven architecture?

Event-driven architecture is a design approach where different parts of a system communicate by producing and reacting to events, rather than calling each other directly. An event might represent something that already happened, like order placed or payment-completed, which other services can listen and react to. This style promotes loose coupling, since services do not need direct knowledge of each other to work together.

Q116 What are the benefits of event-driven architecture?

Event-driven architecture allows services to be added, removed, or updated independently without requiring changes to every other part of the system that depends on them. It also naturally supports asynchronous processing, which can improve responsiveness since services do not need to wait on each other. This flexibility makes it easier to scale and evolve large systems over time.

Q117 What is an event in event-driven architecture?

An event is a record describing something that has already happened in the system, such as a user signing up or an item being added to a cart. Events are typically immutable, meaning once created they are not changed, only reacted to by whichever services are interested. This immutability makes events a reliable, auditable record of everything that has occurred.

Q118 What is a challenge of event-driven architecture?

A major challenge of event-driven architecture is tracing what happened across a system, since a single action might trigger a long, indirect chain of events across many services. This can make debugging harder compared to a simpler, direct request-response system where the flow is easier to follow. Good logging, tracing tools, and clear event naming conventions are essential to managing this added complexity.

Q119 What is a monolithic architecture?

A monolithic architecture is when an entire application is built and deployed as a single, unified codebase, with all its features tightly bundled together. This makes it simple to develop and deploy early on, since there is only one thing to build, test, and run. However, as the application grows, a monolith can become difficult to scale, update, and maintain.

Q120 What is a microservices architecture?

A microservices architecture breaks an application into many small, independent services, each responsible for a specific piece of functionality, like payments or user accounts. These services communicate with each other over the network, usually through APIs, and can be developed, deployed, and scaled independently. This independence offers flexibility, though it adds complexity in coordinating and monitoring many separate pieces.

Q121 What are the advantages of microservices over a monolith?

Microservices allow different teams to work on different services independently, using different technologies if needed, without stepping on each other's code. They also let you scale only the specific services that need more resources, rather than scaling an entire monolithic application at once. This flexibility can lead to faster development cycles and more efficient use of infrastructure as a system grows.

Q122 What are the challenges of microservices?

Microservices introduce challenges like managing network communication between services, handling partial failures gracefully, and keeping data consistent across separate databases. Debugging also becomes harder, since a single user request might touch many different services before completing. Because of this added operational complexity, microservices are usually most beneficial for larger teams and more complex applications, rather than small simple projects.

Q123 What is service discovery?

Service discovery is the process by which services in a distributed system automatically find the network location of other services they need to talk to. In a dynamic environment where servers are constantly starting, stopping, or being replaced, hardcoding IP addresses would quickly become unreliable. Service discovery keeps track of which instances are currently available so requests can always be routed correctly.

Q124 What is the difference between client-side and server-side service discovery?

In client-side service discovery, the calling service directly queries a registry to find available instances and picks one itself before making a request. In server-side service discovery, the calling service simply sends its request to a load balancer or router, which handles looking up and forwarding to an available instance. Serverside discovery keeps client code simpler, while client-side discovery can offer more control over load balancing decisions.

Q125 What is a service registry?

A service registry is a central directory that keeps track of all currently running service instances and their network locations, often updated automatically as instances start and stop. Other services query this registry to find out where to send their requests at any given moment. Popular tools for building service registries include Consul, etcd, and Eureka.

Q126 Why is service discovery especially important in cloud environments?

Service discovery is especially important in cloud environments because servers are frequently created and destroyed automatically as systems scale up and down or recover from failures. Static configuration with fixed IP addresses simply cannot keep up with this constant change. Automated service discovery ensures that traffic always finds currently healthy instances without requiring manual updates.

Q127 What is a container?

A container is a lightweight, self-contained package that bundles an application together with everything it needs to run, such as libraries and configuration. Unlike a full virtual machine, containers share the host operating system's kernel, which makes them much faster to start and more resource efficient. This consistency means an application behaves the same way whether it runs on a developer's laptop or a production server.

Q128 What is Docker?

Docker is the most widely used platform for building, sharing, and running containers, providing simple tools to package applications along with their dependencies. Developers write a Dockerfile describing exactly how to build an image, which can then be run consistently anywhere Docker is installed. This has made Docker a standard tool for ensuring applications run identically across different environments.

Q129 What is the difference between a container and a virtual machine?

A virtual machine includes a full copy of an operating system and is managed by a hypervisor, making it heavier and slower to start. A container shares the host machine's operating system kernel and only packages the application and its dependencies, making it much lighter and faster to launch. Because of this, you can typically run far more containers than virtual machines on the same hardware.

Q130 What is a Docker image?

A Docker image is a read-only template that contains everything needed to run an application, including code, runtime, libraries, and settings. When you run an image, it becomes a live, running container based on that template. Images can be shared through registries, allowing teams to reliably distribute the exact same application environment to anyone who needs it.

Q131 What is Kubernetes?

Kubernetes is an open-source platform for automatically deploying, scaling, and managing containerized applications across a cluster of machines. Instead of manually starting and monitoring containers on individual servers, Kubernetes handles scheduling, restarting failed containers, and distributing load automatically. It has become the standard tool for running containers reliably at scale in production.

Q132 What is a pod in Kubernetes?

A pod is the smallest deployable unit in Kubernetes, typically containing one or more tightly coupled containers that share the same network and storage. Most commonly a pod runs a single container, but sometimes a helper container is included alongside the main one, such as for logging. Kubernetes manages pods rather than individual containers directly, scheduling them onto available machines in the cluster.

Q133 What does container orchestration mean?

Container orchestration means automatically managing the lifecycle of many containers, including starting them, restarting failed ones, scaling them up or down, and distributing them across machines. Without orchestration, engineers would need to manually track and manage every container running across a fleet of servers, which quickly becomes impossible at scale. Kubernetes is the most popular orchestration tool used for this purpose today.

Q134 How does Kubernetes help with self-healing systems?

Kubernetes continuously monitors the health of running containers and automatically restarts or replaces any that crash or stop responding correctly. If an entire server in the cluster fails, Kubernetes can automatically reschedule the affected containers onto healthy machines. This self-healing behavior reduces the need for manual intervention when small failures occur, keeping the overall system more resilient.

Q135 What does CI/CD stand for?

CI/CD stands for Continuous Integration and Continuous Delivery, or sometimes Continuous Deployment, referring to automated processes for building, testing, and releasing software. Continuous integration focuses on automatically testing and merging code changes frequently, while continuous delivery or deployment focuses on automatically releasing those changes to users. Together, they help teams ship software faster and with fewer manual errors.

Q136 What is continuous integration?

Continuous integration is the practice of frequently merging small code changes into a shared codebase, with automated tests running each time to catch problems early. Instead of waiting weeks to combine everyone's work, developers integrate constantly, which makes conflicts and bugs much easier to spot and fix. This steady rhythm of testing keeps the codebase in a consistently working state.

Q137 What is the difference between continuous delivery and continuous deployment?

Continuous delivery means code changes are automatically tested and prepared for release, but a human still manually approves the final deployment to production. Continuous deployment goes one step further and automatically releases every change that passes all tests, with no manual approval step at all. Many teams choose continuous delivery for extra safety, while others fully automate with continuous deployment once they trust their testing thoroughly.

Q138 Why are automated pipelines important in modern software development?

Automated pipelines remove the need for manual, error-prone steps when testing and deploying software, making releases faster and more consistent. They catch bugs early by running tests automatically on every change, long before that code ever reaches real users. This automation lets teams release new features and fixes far more frequently and confidently than manual processes would allow.

Q139 What is a SQL database?

A SQL database stores data in structured tables with predefined columns and strict relationships between them, and it is queried using Structured Query Language. Because of this rigid structure, SQL databases are excellent at enforcing data consistency and handling complex queries that join multiple related tables. Popular examples include PostgreSQL, MySQL, and Microsoft SQL Server.

Q140 What is a NoSQL database?

A NoSQL database stores data in more flexible formats, such as documents, keyvalue pairs, or graphs, instead of rigid tables with fixed columns. This flexibility makes it easier to handle rapidly changing or unstructured data and often allows for easier horizontal scaling across many servers. Popular examples include MongoDB for documents, Redis for key-value data, and Cassandra for wide-column storage.

Q141 When would you choose SQL over NoSQL?

You would choose SQL when your data has clear, stable relationships and you need strong consistency guarantees, such as in banking or financial applications. SQL databases are also a strong choice when complex queries involving multiple related tables are common, since SQL is specifically built for that kind of querying. Applications that prioritize strict data integrity over raw scaling flexibility usually lean toward SQL.

Q142 When would you choose NoSQL over SQL?

You would choose NoSQL when your data does not fit neatly into fixed tables, changes shape frequently, or needs to scale horizontally across many servers easily. NoSQL databases are common in applications with huge volumes of semistructured data, like user activity logs or product catalogs with varying attributes. The tradeoff is often slightly weaker consistency guarantees in exchange for greater flexibility and scalability.

Q143 What is a database index?

A database index is a special data structure that allows the database to find rows matching a query much faster, without scanning every single row in a table. It works similarly to the index at the back of a book, letting you jump directly to relevant pages instead of reading the entire book. Indexes dramatically speed up read queries, though they come with some tradeoffs.

Q144 How does an index speed up queries?

An index organizes data in a way, often using a structure called a B-tree, that lets the database quickly narrow down which rows match a search condition. Instead of checking every row one by one, the database can jump almost directly to the relevant section of data. This can turn a search that would take seconds on a huge table into one that takes milliseconds.

Q145 What is the tradeoff of adding too many indexes?

While indexes speed up reads, each additional index also slows down writes, since the database must update every relevant index whenever data is inserted, updated, or deleted. Indexes also consume extra storage space, which can add up significantly on very large tables. Because of this, engineers usually add indexes carefully, focusing only on columns that are frequently searched or filtered on.

Q146 What is a composite index?

A composite index is an index built across multiple columns together, rather than just one, which can speed up queries that filter or sort by those columns in combination. For example, an index on both last name and first name together would speed up searches that use both fields at once. The order of columns in a composite index matters significantly for how effectively it can be used.

Q147 What is database normalization?

Database normalization is the process of organizing data in a relational database to reduce redundancy and improve data integrity, typically by splitting data into multiple related tables. Instead of repeating the same customer information in every order record, normalization stores it once and references it by an ID. This reduces the risk of inconsistent or duplicated data scattered throughout the database.

Q148 What is denormalization and why would you use it?

Denormalization is the intentional process of combining data back together, sometimes duplicating it across tables, in order to improve read performance by avoiding expensive joins. It sacrifices some storage efficiency and consistency guarantees in exchange for faster queries, which can matter a lot in high-traffic, read-heavy systems. Many large-scale systems denormalize strategically after starting with a fully normalized design.

Q149 What problem does normalization solve?

Normalization solves the problem of data anomalies, where updating information in one place but forgetting to update it elsewhere leads to inconsistent, contradictory data. For example, if a customer's address is stored in ten different order records, updating just one leaves the others out of date. By storing that address in a single dedicated table, an update only ever needs to happen in one place.

Q150 What is a foreign key?

A foreign key is a column in one table that references the primary key of another table, creating a link between related pieces of data. For example, an orders table might have a `customer_id` foreign key that points back to a specific row in the customers table. Foreign keys help enforce relationships and prevent orphaned records that reference data which no longer exists.

Q151 What is database sharding?

Database sharding is the practice of splitting a large database into smaller, separate pieces called shards, each stored on a different server. This allows a system to handle far more data and traffic than a single server could manage alone, since the load is spread across many machines. Each shard typically holds a distinct subset of the overall data, such as users whose ID falls within a certain range.

Q152 How do you decide how to shard a database?

Deciding how to shard a database usually involves choosing a shard key, such as user ID or region, that determines which shard a particular piece of data belongs to. A good shard key spreads data and traffic evenly across shards, avoiding a situation where one shard receives far more load than the others. Poor shard key choices can lead to uneven, unbalanced systems that defeat the purpose of sharding.

Q153 What is a hot shard?

A hot shard is a shard that receives a disproportionately large amount of traffic or data compared to other shards, often due to a poorly chosen shard key. For example, if all requests for a hugely popular celebrity account land on the same shard, that shard can become overwhelmed while others sit idle. Careful shard key selection and monitoring help identify and prevent hot shards before they cause performance problems.

Q154 What are the challenges of sharding?

Sharding introduces challenges like handling queries that need data from multiple shards at once, rebalancing data when shards become uneven, and maintaining consistency across separate pieces. It also adds operational complexity, since there are now many databases to manage, monitor, and back up instead of just one. Because of this complexity, sharding is usually introduced only once a single database can no longer handle the load.

Q155 What is database replication?

Database replication is the process of copying data from one database server to one or more other servers, so multiple copies of the same data exist. This improves both reliability, since a backup copy exists if the primary server fails, and performance, since read requests can be spread across multiple copies. Replication is a fundamental building block for making databases more resilient and scalable.

Q156 What is the difference between synchronous and asynchronous replication?

Synchronous replication waits for a write to be confirmed on all replica servers before considering the write complete, which guarantees consistency but can add delay. Asynchronous replication confirms the write immediately on the primary server and copies it to replicas afterward, which is faster but risks a small window of data loss if the primary fails before replicating. Many systems choose asynchronous replication for performance, accepting the small consistency tradeoff.

Q157 Why is replication important for read-heavy applications?

Replication is important for read-heavy applications because it lets you distribute read queries across multiple replica servers instead of overloading a single database with all the traffic. This can massively increase the number of reads a system can handle simultaneously, since each replica can serve requests independently. Writes are usually still sent to a single primary server to keep data consistent.

Q158 What is replication lag?

Replication lag is the delay between when data is written to the primary database and when that same data becomes visible on a replica server. During this brief window, a replica might return slightly outdated information if queried immediately after a write. Applications sensitive to this delay may need to read from the primary directly for certain critical, just-written data.

Q159 What is master-slave replication?

In master-slave replication, all write operations go to a single primary, or master, server, which then copies those changes out to one or more slave, or replica, servers. Slaves typically only handle read requests, never accepting writes directly, which keeps the data flow simple and predictable. This setup is straightforward to reason about, though the master can become a bottleneck for write-heavy workloads.

Q160 What is master-master replication?

In master-master replication, two or more servers can each accept write operations, and changes are synchronized between them in both directions. This allows write traffic to be spread across multiple servers instead of relying on just one, improving write scalability and availability. However, it introduces the risk of write conflicts if the same piece of data is modified differently on two masters at nearly the same time.

Q161 What is a write conflict in master-master replication?

A write conflict occurs when the same piece of data is updated differently on two different master servers before they have a chance to synchronize with each other. The system then has to decide how to resolve the disagreement, often using rules like last write wins or custom conflict resolution logic. These conflicts are a key reason why master-master setups are more complex to manage than simpler master-slave configurations.

Q162 When would you choose master-master over master-slave replication?

You would choose master-master replication when you need to accept writes from multiple geographic regions with low latency, since each region can write to its own local master. Master-slave is simpler and works well when all writes can reasonably flow through a single, central primary server. The added complexity of conflict resolution in master-master setups is only worth it when write availability across multiple locations is truly essential.

Q163 What is consistent hashing?

Consistent hashing is a technique for distributing data across a set of servers in a way that minimizes how much data needs to move when servers are added or removed. It maps both servers and data keys onto a conceptual ring, and each key is assigned to the nearest server going clockwise around that ring. This approach avoids the massive reshuffling that simpler hashing methods would require whenever the number of servers changes.

Q164 Why is consistent hashing better than simple modulo hashing for distributed systems?

Simple modulo hashing, like $\text{key hash} \bmod \text{number of servers}$, causes nearly all keys to move to different servers whenever a server is added or removed. Consistent hashing instead only requires a small fraction of keys to move, since each server only affects the small portion of the ring immediately before it. This makes scaling a distributed cache or database far less disruptive.

Q165 What problem does consistent hashing solve?

Consistent hashing solves the problem of minimizing data movement and cache invalidation when the number of servers in a distributed system changes. Without it, adding or removing even a single server could require remapping almost every piece of data, causing a huge spike in load and cache misses. This makes consistent hashing especially valuable for distributed caches and databases that need to scale up or down smoothly.

Q166 Where is consistent hashing commonly used?

Consistent hashing is commonly used in distributed caching systems, like memcached clusters, and in distributed databases such as Cassandra and DynamoDB, to decide which server should store a given piece of data. It is also used in load balancers to consistently route the same client to the same backend server whenever possible. Its ability to gracefully handle changing cluster sizes makes it a foundational tool in large distributed systems.

Q167 What is the CAP theorem?

The CAP theorem states that a distributed system can only guarantee two out of three properties at the same time: Consistency, Availability, and Partition tolerance. Since network partitions are essentially unavoidable in real distributed systems, engineers usually have to choose between prioritizing consistency or availability when a partition occurs. Understanding this tradeoff helps guide major decisions about how a distributed database or service should behave under failure.

Q168 What does consistency mean in the CAP theorem?

In the CAP theorem, consistency means that every node in a distributed system returns the same, most recent data for a given request, no matter which node is queried. If one server has just written new data, every other server should reflect that same update immediately when asked. Achieving strict consistency across many distributed nodes usually requires extra coordination, which can slow things down.

Q169 What does availability mean in the CAP theorem?

In the CAP theorem, availability means every request to the system receives a response, even if that response might not reflect the absolute latest data. An available system prioritizes always giving an answer over always giving the perfectly up to date answer. This tradeoff is often acceptable for systems like social media feeds, where slightly stale data is far less harmful than the app appearing broken.

Q170 What does partition tolerance mean in the CAP theorem?

Partition tolerance means the system continues to operate even if network failures prevent some nodes from communicating with others. Since real networks do occasionally experience failures or delays, most distributed systems are designed to be partition tolerant by necessity. Given a partition, a system must then choose to sacrifice either full consistency or full availability, since it cannot perfectly guarantee both.

Q171 What does ACID stand for?

ACID stands for Atomicity, Consistency, Isolation, and Durability, a set of properties that guarantee reliable transactions in traditional relational databases. Atomicity ensures a transaction either fully completes or fully fails with no partial changes, while consistency ensures data always follows defined rules. Isolation keeps concurrent transactions from interfering with each other, and durability ensures completed transactions survive even a crash.

Q172 What does BASE stand for?

BASE stands for Basically Available, Soft state, and Eventually consistent, describing a more relaxed set of guarantees often used in large distributed NoSQL systems. Instead of strict, immediate consistency, BASE systems accept that data might be temporarily inconsistent but will eventually settle into a consistent state. This tradeoff allows BASE systems to prioritize availability and scalability over strict correctness at every single moment.

Q173 How do ACID and BASE differ in their approach to consistency?

ACID systems prioritize strict, immediate consistency, ensuring that every read always reflects the most recent completed write, even if that means sacrificing some availability during failures. BASE systems accept temporary inconsistency in exchange for staying highly available and scalable, trusting that data will converge to a consistent state eventually. Choosing between them depends on whether your application needs perfect accuracy immediately or can tolerate brief periods of staleness.

Q174 When would BASE be preferred over ACID?

BASE is often preferred over ACID in large-scale distributed systems where availability and partition tolerance matter more than perfect, instant consistency, such as social media feeds or product catalogs. Users browsing a shopping site generally will not notice or mind if inventory counts are a few seconds out of date. ACID remains essential for use cases like financial transactions, where even a brief inconsistency could cause serious problems.

Q175 What is eventual consistency?

Eventual consistency is a consistency model where, given enough time without new updates, all replicas of a piece of data will eventually converge to the same value. In the meantime, different nodes might briefly return slightly different, outdated versions of the data. This model favors availability and performance over the strict, immediate consistency guaranteed by traditional relational databases.

Q176 What is an example of eventual consistency in everyday systems?

A common example is a social media like count, where you might briefly see slightly different numbers on different devices right after liking a post, before everything syncs up. Similarly, when you update your profile picture, some of your friends might see the old picture for a few seconds while the update propagates. In both cases, the system prioritizes staying fast and available over instantly syncing every single node.

Q177 What are the tradeoffs of eventual consistency?

Eventual consistency allows a system to remain highly available and responsive, even during network issues, since nodes do not have to wait for full synchronization before responding. The tradeoff is that users might occasionally see stale or slightly inconsistent data for a brief period after an update. Applications must be designed to tolerate or gracefully handle this temporary inconsistency where it matters less.

Q178 How do systems eventually resolve conflicting updates?

Systems using eventual consistency often use techniques like timestamps, version vectors, or last-write-wins rules to determine which update should ultimately be treated as correct. Some systems even involve the application logic itself in resolving conflicts, especially when merging data intelligently makes more sense than simply picking one version. The overall goal is that, once all updates have propagated, every node ends up agreeing on the same final state.

Q179 What is data partitioning?

Data partitioning is the general practice of splitting a large dataset into smaller, more manageable pieces, often distributed across multiple servers or storage locations. It is closely related to sharding but can also apply within a single database to improve query performance on very large tables. Good partitioning strategies aim to keep data access patterns efficient and evenly distributed.

Q180 What is range-based partitioning?

Range-based partitioning divides data into partitions based on ranges of values, such as splitting users alphabetically by last name or orders by date ranges. It is intuitive and works well for queries that naturally filter by range, like fetching all orders from a specific month. The downside is that certain ranges can become unevenly popular, leading to hot spots on particular partitions.

Q181 What is hash-based partitioning?

Hash-based partitioning applies a hash function to a key, like a user ID, and uses the result to decide which partition the data belongs to. This tends to distribute data much more evenly across partitions compared to range-based partitioning, avoiding hot spots. The tradeoff is that it becomes harder to efficiently query a range of related values, since they end up scattered across different partitions.

Q182 What is directory-based partitioning?

Directory-based partitioning uses a separate lookup service, or directory, that keeps track of exactly which partition holds each piece of data. This offers a lot of flexibility, since the mapping between data and partitions can be changed without needing a strict mathematical formula. The tradeoff is that the lookup directory itself becomes an additional component that must be kept fast and highly available.

Q183 What is a CDN?

A CDN, or Content Delivery Network, is a geographically distributed network of servers that cache and deliver content, like images, videos, and web pages, from a location close to the user. Instead of every request traveling all the way back to a single origin server, users are served by a nearby CDN server, drastically reducing load time. This is especially valuable for websites with users spread across many different countries.

Q184 How does a CDN reduce latency?

A CDN reduces latency by storing copies of content at edge servers positioned physically closer to end users around the world. Since data has less physical distance to travel, requests are answered faster than if every single request had to reach one central server far away. This distance reduction is one of the simplest yet most effective ways to speed up a global application.

Q185 What kind of content is commonly served through a CDN?

CDNs commonly serve static content like images, videos, CSS and JavaScript files, and downloadable files, since these do not change frequently and are safe to cache. Some modern CDNs also cache dynamic content or provide edge computing capabilities to run logic closer to users. Serving static assets through a CDN also reduces load on the origin server, freeing it up to handle more complex requests.

Q186 What happens when requested content is not yet cached at a CDN edge server?

When requested content is not yet cached at a CDN edge server, the CDN typically fetches it from the origin server once, a process known as a cache miss, and stores a copy for future requests. Subsequent requests for the same content from nearby users are then served directly from the cache, called a cache hit, without contacting the origin again. This caching behavior means the very first request in a region is often slightly slower than later ones.

Q187 What is write-through caching?

Write-through caching writes data to both the cache and the underlying database at the same time, ensuring the cache always stays in sync with the source of truth. This guarantees strong consistency between the cache and database, though it can be slightly slower since every write touches both locations. It is a good choice when data accuracy is more important than absolute write speed.

Q188 What is write-back caching?

Write-back caching writes data to the cache first and only updates the underlying database later, often in a batch or after a delay. This can make writes feel much faster to the user, since they do not have to wait for the slower database write to complete immediately. The tradeoff is a higher risk of data loss if the cache fails before those pending writes are saved to the database.

Q189 What is write-around caching?

Write-around caching writes data directly to the database, bypassing the cache entirely, and only populates the cache later when that data is actually read. This avoids filling the cache with data that might never be read again, which can be useful for write-heavy workloads with less predictable read patterns. The downside is that a read immediately after a write will result in a cache miss, since the cache was never populated during the write.

Q190 How do you choose between these caching strategies?

Choosing a caching strategy depends on your application's balance of reads versus writes and how tolerant it is of potential data loss or staleness.

Write-through suits applications that prioritize consistency, write-back suits applications that prioritize fast write performance, and write-around suits workloads with many writes that are rarely read right away. Many real systems even combine different strategies for different types of data within the same application.

Q191 What is cache eviction?

Cache eviction is the process of removing older or less useful data from a cache to make room for new data, since caches always have limited storage space. Different eviction policies decide exactly which piece of data gets removed when the cache becomes full. Choosing the right eviction policy can significantly affect how often the cache actually helps improve performance.

Q192 What is the LRU eviction policy?

LRU, or Least Recently Used, is an eviction policy that removes the piece of data that has not been accessed for the longest amount of time. The assumption behind LRU is that data used recently is more likely to be used again soon, so keeping it in the cache is more valuable. LRU is one of the most commonly used and intuitive cache eviction strategies in practice.

Q193 What is the LFU eviction policy?

LFU, or Least Frequently Used, is an eviction policy that removes the piece of data that has been accessed the fewest number of times overall. This differs from LRU by focusing on total usage frequency rather than how recently something was accessed. LFU can perform better in situations where certain data is consistently popular over a long period, rather than just recently popular.

Q194 What is a TTL and how does it relate to caching?

TTL, or Time To Live, is a set duration after which a piece of cached data is automatically considered expired and removed or refreshed. It ensures that cached data does not become permanently stale by forcing periodic updates from the original source. Choosing an appropriate TTL is a balancing act between keeping data fresh and reducing unnecessary load on the underlying system.

Q195 What is a distributed cache?

A distributed cache spreads cached data across multiple servers instead of storing it all on a single machine, allowing the cache to handle much larger amounts of data and traffic. This is essential for large applications where a single server's memory would not be nearly enough to hold all frequently accessed data. Popular distributed caching tools include Redis and Memcached.

Q196 What is Redis commonly used for?

Redis is an in-memory data store commonly used as a fast cache, but it also supports more advanced data structures like lists, sets, and sorted sets, and can be used for tasks like rate limiting or session storage. Because it keeps data in memory, Redis can serve requests in sub-millisecond time, far faster than typical disk-based databases. Its versatility has made it one of the most popular caching and data storage tools in modern system design.

Q197 What is Memcached?

Memcached is a simple, high-performance, distributed memory caching system designed specifically for caching small chunks of data, like the results of database queries or rendered page fragments. It is intentionally simpler than Redis, focusing purely on being an extremely fast key-value cache without extra data structures. Many systems choose Memcached when they need straightforward caching without the additional features Redis provides.

Q198 Why might a distributed cache improve overall system performance so significantly?

A distributed cache keeps frequently accessed data in fast memory across multiple servers, avoiding the need to repeatedly query a much slower database or recompute expensive results. Since memory access is dramatically faster than disk access, even a small distributed cache can absorb a huge percentage of read traffic. This can reduce database load enough to let the same backend infrastructure support far more users.

Q199 What is round robin load balancing?

Round robin load balancing distributes incoming requests evenly across a list of servers by cycling through them one at a time in a fixed order. It is simple to implement and works reasonably well when all servers have similar capacity and each request takes about the same amount of time to process. It does not account for servers that might already be under heavier load than others, though.

Q200 What is least connections load balancing?

Least connections load balancing sends each new request to whichever server currently has the fewest active connections at that moment. This approach adapts better to situations where some requests take much longer than others, since it avoids piling more work onto an already busy server. It requires the load balancer to actively track connection counts across all servers in real time.

Q201 What is weighted load balancing?

Weighted load balancing assigns different servers a weight based on their capacity, sending proportionally more traffic to more powerful servers and less to weaker ones. This is useful in situations where a server pool contains a mix of machine sizes, ensuring stronger servers handle their fair share of the extra load. Weights can be adjusted manually or automatically based on real-time performance metrics.

Q202 What is IP hash load balancing?

IP hash load balancing uses a hash of the client's IP address to consistently route that same client to the same backend server every time. This can be useful when a server needs to remember session information about a specific client locally, without relying on shared, external session storage. The tradeoff is that traffic distribution can become uneven if certain IP addresses generate far more requests than others.

Q203 What is vertical scaling?

Vertical scaling, also called scaling up, means increasing the power of a single server by adding more CPU, memory, or storage instead of adding more servers. It is simple because the application does not need to be redesigned to run across multiple machines. However, there is always a physical limit to how much a single machine can be upgraded, and it creates a single point of failure.

Q204 What is horizontal scaling?

Horizontal scaling, also called scaling out, means adding more servers to handle increased load, rather than making a single server more powerful. This approach can theoretically scale almost indefinitely by continuing to add more machines as demand grows. It requires the application to be designed to work across multiple servers, often needing load balancing and careful handling of shared state.

Q205 What are the tradeoffs between horizontal and vertical scaling?

Vertical scaling is simpler to implement but has hard physical limits and creates a single point of failure if that one powerful server goes down. Horizontal scaling offers far more growth potential and improved fault tolerance, since losing one server out of many is less catastrophic, but it requires more complex architecture. Most large-scale systems eventually rely primarily on horizontal scaling once vertical scaling limits are reached.

Q206 Why is horizontal scaling generally preferred for large systems?

Horizontal scaling is generally preferred for large systems because it avoids the hard ceiling of a single machine's maximum capacity, allowing near-limitless growth by adding more servers. It also improves fault tolerance, since traffic can be redirected away from a failed server to the many others still running. This is why most major internet companies rely heavily on horizontally scaled architectures rather than a few extremely powerful machines.

Q207 What is a stateless service?

A stateless service does not retain any information about previous requests between calls, treating each request as a completely independent transaction. This makes it very easy to scale horizontally, since any server can handle any request without needing shared memory of past interactions. Most modern web APIs are designed to be stateless for exactly this reason.

Q208 What is a stateful service?

A stateful service retains information, or state, about previous interactions, such as a user's session details or an ongoing multiplayer game's current status. This can make certain features easier to implement, but it complicates scaling, since a specific server may need to keep handling requests from the same client. Stateful services often require extra mechanisms like sticky sessions or shared external storage to scale reliably.

Q209 Why is statelessness valuable for scalability?

Statelessness is valuable for scalability because any available server can handle any incoming request, allowing a load balancer to freely distribute traffic without worrying about which server handled previous requests. This flexibility makes it much easier to add or remove servers on demand as traffic changes. Stateful systems, by contrast, need extra coordination to achieve the same level of flexible scaling.

Q210 How can a stateful application be made easier to scale?

A stateful application can be made easier to scale by moving state out of individual servers and into shared external storage, like a database or distributed cache, that any server can access. This effectively turns the application servers themselves back into stateless components, even though the overall system still manages state somewhere. Techniques like sticky sessions can also help, though they are generally considered a less flexible solution.

Q211 What does idempotency mean?

Idempotency means that performing the same operation multiple times produces the same result as performing it just once, with no unintended extra side effects. For example, setting a user's status to active is idempotent, since doing it five times leaves the same end result as doing it once. This property is extremely valuable for building systems that can safely retry failed operations.

Q212 Why is idempotency important when dealing with network retries?

Idempotency is important because networks are unreliable, and a client often cannot be sure whether a request actually succeeded or simply timed out before receiving a response. If an operation is idempotent, the client can safely retry it without worrying about accidentally causing duplicate effects, like charging a customer twice. Without idempotency, retries become risky and can introduce serious bugs.

Q213 What is an example of a non-idempotent operation?

A classic example of a non-idempotent operation is incrementing a value by one, since running it multiple times produces a different result each time, unlike setting a value directly. Similarly, submitting a payment request without any safeguards could charge a customer multiple times if the request is accidentally retried. Designing operations to be idempotent, or adding safeguards like unique request IDs, helps avoid these problems.

Q214 How can you make a non-idempotent operation safe to retry?

A common technique is to attach a unique idempotency key to each request, allowing the server to recognize and ignore duplicate requests that share the same key. The server can store which keys it has already processed and simply return the original result if the same key is seen again. This pattern is widely used in payment systems to safely handle retries without risking duplicate charges.

Q215 What is a consensus algorithm?

A consensus algorithm is a method that allows multiple distributed nodes to agree on a single value or decision, even when some nodes might fail or messages might be delayed. This agreement is essential for keeping distributed systems consistent, such as agreeing on which node is currently the leader or what the next entry in a log should be. Without consensus, distributed nodes could end up in conflicting, contradictory states.

Q216 What is Paxos?

Paxos is one of the earliest and most well-known consensus algorithms, designed to help distributed nodes agree on a value even in the presence of failures. It is known for being theoretically solid but notoriously difficult to fully understand and implement correctly. Because of this complexity, many newer systems have moved toward alternative algorithms that are easier to reason about.

Q217 What is Raft?

Raft is a consensus algorithm designed specifically to be easier to understand and implement than Paxos, while providing similar guarantees. It works by electing a single leader node responsible for coordinating updates, with other nodes following that leader's decisions. Raft has become popular in many modern distributed systems, including tools like etcd and Consul, largely due to its relative simplicity.

Q218 Why do distributed systems need consensus algorithms at all?

Distributed systems need consensus algorithms because individual nodes can fail, restart, or become temporarily unreachable, yet the overall system still needs to agree on a single, consistent view of critical decisions. Without consensus, different nodes might disagree about something important, like who the current leader is, leading to conflicting or corrupted data. Consensus algorithms provide mathematically proven ways to reach reliable agreement despite these real-world failure conditions.

Q219 What is leader election?

Leader election is the process by which nodes in a distributed system choose a single node to act as the coordinator, or leader, responsible for making certain decisions. Having one clear leader avoids conflicting instructions that might occur if multiple nodes all tried to coordinate independently. If the current leader fails, the remaining nodes automatically elect a new one to take over.

Q220 Why is leader election important in distributed systems?

Leader election is important because many distributed tasks, like assigning work or coordinating writes, are much simpler and safer when handled by a single, agreed-upon leader rather than multiple competing nodes. It helps avoid conflicting decisions that could arise if several nodes acted independently without any coordination. Automatic leader election also ensures the system can recover on its own if the current leader unexpectedly fails.

Q221 What happens if a leader node fails?

If a leader node fails, the remaining nodes in the system detect the failure, usually through missed heartbeat signals, and trigger a new leader election process. A new leader is chosen from the remaining healthy nodes, and the system resumes normal operation once the new leader is established. This automatic recovery is essential for keeping distributed systems available even when individual nodes fail unexpectedly.

Q222 What is a heartbeat in the context of leader election?

A heartbeat is a small, periodic signal that nodes send to each other to confirm they are still alive and functioning correctly. If a leader stops sending heartbeats within an expected time window, other nodes assume it has failed and begin the process of electing a new leader. This simple mechanism is a common way distributed systems detect failures quickly and reliably.

Q223 What is a distributed lock?

A distributed lock is a mechanism that ensures only one process across an entire distributed system can access a particular resource or perform a particular action at a time. This prevents conflicting operations, like two different servers accidentally processing the exact same task simultaneously. Distributed locks are often implemented using tools like Redis or Zookeeper, which can coordinate locking across multiple machines.

Q224 Why are distributed locks harder to implement than regular locks?

Regular locks within a single program can rely on shared memory and simple, fast operations that are guaranteed to work correctly. Distributed locks must instead coordinate across separate machines over an unreliable network, where messages can be delayed, lost, or arrive out of order. This makes correctly implementing distributed locks significantly more complex, requiring careful handling of timeouts and failure scenarios.

Q225 What can go wrong if a distributed lock is not released properly?

If a distributed lock is not released properly, perhaps because the holding process crashed before releasing it, other processes could be permanently blocked from accessing that resource. To prevent this, distributed locks are usually given an expiration time, so they are automatically released even if the original holder fails to release it manually. Careful lock timeout design is essential to balance safety against accidentally releasing a lock too early.

Q226 What is a common tool used to implement distributed locks?

Redis is a common tool used to implement distributed locks, thanks to its fast, atomic operations that can safely check and set a lock value across multiple clients. Zookeeper is another popular choice, specifically designed with strong consistency guarantees well suited for coordination tasks like locking. The right choice often depends on what consistency guarantees the specific use case requires.

Q227 What is authentication?

Authentication is the process of verifying that someone is who they claim to be, typically by checking a username and password or another credential like a fingerprint. It answers the basic question, are you actually who you say you are. Without proper authentication, anyone could pretend to be any user in a system.

Q228 What is authorization?

Authorization is the process of determining what an already authenticated user is allowed to do within a system, such as which pages they can view or actions they can perform. It answers the question, now that we know who you are, what are you allowed to access. Authentication always happens first, since a system needs to know who someone is before deciding what they can do.

Q229 How are authentication and authorization different in practice?

Authentication happens once, typically at login, confirming a user's identity through credentials like a password or token. Authorization happens continuously afterward, checking on every relevant action whether that specific, already identified user has permission to do it. A logged-in user, for example, might be authenticated successfully but still be denied authorization to access another user's private data.

Q230 What is role-based access control?

Role-based access control, or RBAC, is a common authorization approach where permissions are granted to roles, like admin or editor, rather than to individual users directly. Users are then simply assigned one or more roles, automatically inheriting whatever permissions those roles include. This makes managing permissions much simpler, since you can update a role's permissions once instead of updating every individual user.

Q231 What is OAuth?

OAuth is an open standard protocol that allows applications to access a user's data on another service without ever needing to see or store that user's password. For example, OAuth lets you log into a new app using your existing Google account, granting limited access without sharing your Google password directly. It has become the standard way third-party applications request access to user data securely.

Q232 What is a JWT?

A JWT, or JSON Web Token, is a compact, digitally signed piece of data commonly used to represent authenticated user information that can be safely passed between systems. Because it is signed, a server can verify a JWT has not been tampered with, without needing to look up session data in a database on every request. This makes JWTs popular for stateless authentication in modern web applications.

Q233 How does a JWT enable stateless authentication?

A JWT contains all the necessary information about a user's identity and permissions directly within the token itself, digitally signed so it cannot be altered undetected. This means a server can verify and trust the token's contents just by checking its signature, without needing to store and look up session information anywhere. This avoids the need for shared session storage, making it easier to scale authentication across many stateless servers.

Q234 What is a refresh token?

A refresh token is a long-lived credential used to obtain new short-lived access tokens without requiring the user to log in again with their password. Access tokens are usually kept short-lived for security, so if one is stolen, it becomes useless quickly, while the refresh token remains safely stored to request new ones. This pattern balances strong security with a smooth, uninterrupted experience for the user.

Q235 What is encryption?

Encryption is the process of converting readable data into a scrambled, unreadable format that can only be converted back with the correct key, protecting information from unauthorized access. It ensures that even if data is intercepted or stolen, it remains meaningless without the proper key to decrypt it. Encryption is fundamental to protecting sensitive information both while it travels across a network and while it sits in storage.

Q236 What is the difference between symmetric and asymmetric encryption?

Symmetric encryption uses the exact same key to both encrypt and decrypt data, making it fast but requiring a secure way to share that key between parties beforehand. Asymmetric encryption uses a pair of mathematically linked keys, a public key for encrypting and a private key for decrypting, removing the need to share a secret key directly. Many real-world systems combine both, using asymmetric encryption to safely exchange a symmetric key, then using that faster symmetric key for the actual data.

Q237 What is encryption in transit versus encryption at rest?

Encryption in transit protects data while it travels across a network, such as between a browser and a server, commonly implemented using TLS in HTTPS connections. Encryption at rest protects data while it is stored on disk, ensuring that even someone with direct access to the storage cannot read it without the proper key. A well-secured system typically applies both types of encryption to protect data at every stage.

Q238 Why is hashing different from encryption?

Hashing converts data into a fixed-length string in a one-way process that cannot be reversed back into the original data, unlike encryption which is designed to be reversible with the correct key. Hashing is commonly used for storing passwords, since the system only needs to verify a password matches, never needing to recover the original text. Encryption, by contrast, is used whenever the original data genuinely needs to be recovered later.

Q239 What is the principle of least privilege?

The principle of least privilege means giving users, services, and systems only the minimum level of access they truly need to perform their job, nothing more. This limits the potential damage if an account or service is ever compromised, since an attacker would only gain access to a narrowly limited set of resources. It is one of the most fundamental and widely recommended practices in security design.

Q240 Why should sensitive data never be stored in plain text?

Sensitive data like passwords should never be stored in plain text because anyone who gains unauthorized access to the database would immediately be able to read and misuse it directly. Instead, sensitive data is typically hashed or encrypted, so even a data breach does not immediately expose usable information. This extra layer of protection significantly reduces the damage caused by a potential security breach.

Q241 What is defense in depth?

Defense in depth is a security strategy that layers multiple independent protective measures throughout a system, rather than relying on any single point of defense. If one layer, like a firewall, fails or is bypassed, other layers, like encryption or access controls, still provide protection. This layered approach makes it significantly harder for an attacker to compromise an entire system with just one successful exploit.

Q242 Why is input validation important for security?

Input validation is important because it prevents malicious or malformed data from a user from causing unintended behavior, such as SQL injection or cross-site scripting attacks. By checking that incoming data matches expected formats and rejecting anything suspicious, a system closes off many common avenues attackers try to exploit. Proper input validation should always happen on the server side, since client-side checks can easily be bypassed.

Q243 What is logging in system design?

Logging is the practice of recording events that happen within a system, such as errors, warnings, or important actions, into a structured or unstructured record for later review. Logs help engineers understand exactly what happened leading up to a problem, acting like a detailed diary of system behavior. Good logging practices make debugging issues in production dramatically easier.

Q244 What are metrics in system design?

Metrics are numerical measurements collected over time that describe a system's health and performance, such as request count, error rate, or average response time. Unlike detailed logs, metrics are usually aggregated and summarized, making them ideal for spotting trends and setting up automated alerts. Dashboards built from metrics give engineers a quick, high-level view of how a system is currently performing.

Q245 What is distributed tracing?

Distributed tracing tracks a single request as it travels through multiple services in a distributed system, recording how long each step takes along the way. This makes it possible to pinpoint exactly which service is causing a slowdown when a request passes through many different microservices before completing. Without tracing, diagnosing performance problems across many interacting services would be extremely difficult.

Q246 How do logging, metrics, and tracing work together?

Logging, metrics, and tracing together form what is often called the three pillars of observability, each providing a different lens into system behavior. Metrics quickly reveal that something is wrong, like a spike in errors, logs provide detailed context about what specifically happened, and tracing shows exactly where in a complex request flow the problem occurred. Using all three together gives engineers a complete picture for diagnosing issues quickly.

Q247 What is monitoring in system design?

Monitoring is the continuous process of observing a system's health and performance in real time, using collected metrics, logs, and other signals. It allows engineers to notice problems quickly, often before they seriously affect users, rather than discovering issues only after receiving complaints. Good monitoring is essential infrastructure for running any reliable production system.

Q248 What is alerting?

Alerting is the practice of automatically notifying engineers when a monitored metric crosses a concerning threshold, such as error rates spiking or response times slowing dramatically. Well-designed alerts allow teams to respond to problems quickly, often resolving issues before most users even notice anything went wrong. Poorly designed alerts, on the other hand, can either miss real problems or create so much noise that important warnings get ignored.

Q249 What is alert fatigue?

Alert fatigue happens when engineers receive so many alerts, especially false or low-priority ones, that they start ignoring or dismissing them without carefully checking each one. This is dangerous because a truly critical alert might get lost in the noise of many unimportant ones. Careful alert tuning, focusing only on genuinely actionable issues, helps prevent alert fatigue from undermining an otherwise good monitoring system.

Q250 What is a good practice when setting up alert thresholds?

A good practice is to set alert thresholds based on what genuinely indicates a problem affecting users, rather than reacting to every minor, expected fluctuation in metrics. Alerts should also be actionable, meaning whoever receives them should be able to do something useful in response, rather than just acknowledging noise. Regularly reviewing and tuning alert thresholds over time helps keep a monitoring system genuinely useful rather than overwhelming.

Q251 What is a backup in system design?

A backup is a copy of data stored separately from the original, so that information can be recovered if the primary data is lost, corrupted, or deleted. Backups can be taken on a regular schedule, such as nightly, and stored in a different location to protect against local failures. Without reliable backups, a single serious failure could mean permanently losing important data.

Q252 What is disaster recovery?

Disaster recovery is the overall plan and set of procedures an organization follows to restore systems and data after a major failure, such as a data center outage or serious cyberattack. It goes beyond just having backups, including steps for how quickly systems should be restored and in what order priorities should be handled. A well-tested disaster recovery plan can mean the difference between a minor disruption and a catastrophic business failure.

Q253 What is RTO and RPO?

RTO, or Recovery Time Objective, is the maximum acceptable amount of time a system can be down before it must be restored after a disaster. RPO, or Recovery Point Objective, is the maximum acceptable amount of data loss, measured in time, such as losing at most the last five minutes of data. Together, these two metrics guide how frequently backups should be taken and how quickly recovery systems need to respond.

Q254 Why should backups be tested regularly?

Backups should be tested regularly because a backup that cannot actually be restored successfully provides a false sense of security while offering no real protection. Teams sometimes discover, only during an actual emergency, that their backups were incomplete, corrupted, or simply never worked as expected. Regularly practicing full restoration from backups ensures that when a real disaster strikes, recovery actually works as planned.

Q255 What is a data warehouse?

A data warehouse is a large, centralized repository designed to store and organize data from many different sources, optimized specifically for analysis and reporting rather than everyday transactions. Unlike a typical application database, a data warehouse is built to efficiently run complex queries across huge amounts of historical data. Companies use data warehouses to power business intelligence dashboards and long-term trend analysis.

Q256 What does ETL stand for?

ETL stands for Extract, Transform, and Load, describing the three main steps involved in moving data from source systems into a data warehouse. Extract pulls data out of original sources, transform cleans and reshapes that data into a consistent format, and load places the finished data into its destination. ETL pipelines are essential for combining messy, scattered data into something clean and useful for analysis.

Q257 Why is a data warehouse separate from an application's main database?

A data warehouse is kept separate so that heavy, complex analytical queries do not slow down the main application database that regular users rely on for fast, everyday transactions. It also allows data from many different source systems to be combined into a single, unified view for company-wide reporting. Keeping these concerns separate lets each system be optimized for its own specific purpose.

Q258 What is the difference between ETL and ELT?

ETL transforms data before loading it into the destination system, while ELT loads raw data first and performs the transformation afterward, directly within the destination system. ELT has become more popular as modern data warehouses have become powerful enough to handle heavy transformation work themselves. The right choice often depends on the specific tools and processing power available in a given data pipeline.

Q259 What is batch processing?

Batch processing handles data in large groups, or batches, that are collected over a period of time and then processed all together, rather than immediately as each piece arrives. This approach is efficient for tasks that do not require instant results, such as generating a nightly sales report from an entire day's transactions. Batch jobs are typically scheduled to run at set intervals, like once every hour or once a day.

Q260 What is stream processing?

Stream processing handles data continuously, as individual events arrive in real time, rather than waiting to collect them into larger batches first. This allows systems to react to information almost instantly, which is essential for use cases like fraud detection or live analytics dashboards. Tools like Apache Kafka Streams and Apache Flink are commonly used to build stream processing pipelines.

Q261 When would you choose stream processing over batch processing?

You would choose stream processing when your application genuinely needs to react to data the moment it arrives, such as detecting a fraudulent transaction within seconds rather than hours. Batch processing remains a great choice when immediate results are not necessary, since it is often simpler to build and can be more resource efficient. Some systems use both together, combining real-time stream processing with periodic batch processing for deeper historical analysis.

Q262 What is a tradeoff of stream processing compared to batch processing?

Stream processing systems are generally more complex to build and operate, since they must handle continuous data reliably, in order, and often at very high volume without interruption. Batch processing systems are usually simpler, since they can process a fixed, known amount of data all at once during a scheduled window. This added complexity of stream processing is worth it only when the immediacy of real-time results genuinely matters for the use case.

Q263 What is CQRS?

CQRS, or Command Query Responsibility Segregation, is a design pattern that separates the operations that change data, called commands, from the operations that read data, called queries. This separation allows each side to be optimized and scaled independently, since read and write workloads often have very different performance requirements. CQRS is especially useful in complex systems where reads vastly outnumber writes, or vice versa.

Q264 What is event sourcing?

Event sourcing is a pattern where, instead of just storing the current state of data, a system stores every single change as a sequence of immutable events over time. The current state can always be reconstructed by replaying all the events from the beginning, which provides a complete, auditable history of everything that ever happened. This is powerful for use cases where understanding exactly how data reached its current state truly matters.

Q265 How do CQRS and event sourcing often work together?

CQRS and event sourcing are frequently used together, with event sourcing serving as the write model that records every change as an event, and CQRS providing separately optimized read models built from those events. This combination allows a system to maintain a perfect historical record while also serving fast, efficient reads tailored to specific use cases. Together, they offer powerful flexibility, though at the cost of added architectural complexity.

Q266 What is a downside of event sourcing?

A downside of event sourcing is the added complexity of managing and replaying potentially huge histories of events, especially as a system accumulates data over many years. Debugging can also become more challenging, since understanding current state requires reasoning through a sequence of past events rather than reading a single, simple record. Because of this complexity, event sourcing is usually reserved for systems where its specific benefits, like full auditability, clearly outweigh the added overhead.

Q267 What is serverless computing?

Serverless computing lets developers run code without having to manage the underlying servers directly, since the cloud provider automatically handles provisioning, scaling, and maintenance. Despite the name, servers are still involved behind the scenes, but developers simply write functions and let the platform run them on demand. This model can significantly reduce operational overhead for certain types of workloads.

Q268 What is a serverless function?

A serverless function, sometimes called a cloud function, is a small piece of code that runs in response to a specific trigger, such as an HTTP request or a new file being uploaded. It runs only when needed and automatically scales up or down based on demand, without the developer needing to manage any servers directly. Popular serverless platforms include AWS Lambda, Google Cloud Functions, and Azure Functions.

Q269 What are the benefits of serverless computing?

Serverless computing removes the need to manage or provision servers manually, automatically scales to handle varying levels of traffic, and typically charges only for the actual compute time used. This can be especially cost effective for workloads with unpredictable or infrequent usage patterns, since idle time costs nothing. It also lets developers focus more on writing application logic rather than managing infrastructure.

Q270 What are the downsides of serverless computing?

Serverless functions can experience a short delay, known as a cold start, when they have not run recently and need to be initialized before handling a request. They can also be less cost effective than traditional servers for workloads with constant, high, predictable traffic. Additionally, serverless architectures can make certain patterns, like long-running processes or complex local debugging, more difficult compared to traditional applications.

Q271 What is edge computing?

Edge computing means processing data closer to where it is actually generated or consumed, such as on nearby servers or devices, rather than sending everything back to a distant central data center. This reduces the time it takes for data to travel, which can significantly cut down on latency for time-sensitive applications. It is increasingly used for things like Internet of Things devices, gaming, and video processing.

Q272 How is edge computing different from traditional cloud computing?

Traditional cloud computing typically processes data in large, centralized data centers that might be physically far away from the end user. Edge computing instead pushes some of that processing out to smaller servers located much closer to users or devices, reducing the distance data has to travel. This tradeoff often means less centralized processing power available, in exchange for significantly reduced latency.

Q273 What are common use cases for edge computing?

Common use cases for edge computing include autonomous vehicles needing instant decisions, smart factory sensors processing data locally, and CDNs running logic close to users to speed up dynamic content. Any application where even small amounts of network delay can cause real problems is a strong candidate for edge computing. As more devices become connected, the demand for processing data closer to its source continues to grow.

Q274 How does edge computing relate to CDNs?

Many modern CDNs have expanded beyond simply caching static files to also running small pieces of application logic directly at their edge servers, blurring the line between CDN and edge computing. This lets certain computations, like authentication checks or simple redirects, happen close to the user instead of requiring a round trip to a distant origin server. This evolution has made CDNs an increasingly important part of edge computing strategies.

Q275 What are the core requirements of a URL shortener like Bitly?

A URL shortener needs to take a long URL and generate a short, unique code that redirects to the original address when visited. Core requirements typically include generating unique short codes, storing the mapping between short codes and original URLs, and redirecting users quickly when they visit a short link. Additional features might include tracking click analytics or allowing custom short codes.

Q276 How would you generate unique short codes for a URL shortener?

One common approach is to use a counter that increases with every new URL, then convert that number into a short string using a larger character set like base 62, which includes letters and digits. Another approach is generating a random string and checking the database to ensure it has not already been used. The counterbased approach guarantees uniqueness without extra checks, while the random approach is simpler but requires handling occasional collisions.

Q277 What database considerations matter for a URL shortener?

A URL shortener needs extremely fast lookups when redirecting, since every short link click depends on quickly finding the original URL, which often means using a key-value store or a heavily indexed database. Given how read-heavy this system is, with far more redirects than new URLs created, a distributed cache in front of the database can dramatically improve performance. The write side is comparatively simple, since it primarily just needs to insert new mappings.

Q278 How would you scale a URL shortener to handle massive traffic?

To scale a URL shortener, you would add caching for popular short links, since a small percentage of URLs typically account for a large percentage of total clicks. You would also use a CDN or geographically distributed servers to reduce latency for the redirect step, since speed really matters for a good user experience. Database sharding based on the short code can help scale storage as the number of shortened URLs grows into the billions.

Q279 What are the core components of a chat application like WhatsApp?

The core components of a chat application typically include a way for clients to establish real-time connections, often WebSockets, a message storage system, and a service to handle delivering messages to online or offline users. It also needs mechanisms to track which users are currently online and to reliably deliver messages once a user comes back online after being offline. Group chats add additional complexity around managing membership and delivering to many recipients at once.

Q280 How does a chat app handle message delivery for offline users?

When a recipient is offline, the chat app typically stores the message in a database queue tied to that user, marking it as undelivered. Once the user reconnects, perhaps by opening the app or establishing a new WebSocket connection, the server checks for any pending messages and delivers them immediately. This ensures messages are never lost, even if the recipient was not actively connected when they were sent.

Q281 Why are WebSockets a natural fit for a chat application?

WebSockets are a natural fit because chat applications need instant, two-way communication, letting both the sender and receiver exchange messages the moment they happen without repeated polling. A persistent WebSocket connection means new messages can be pushed to a user's device immediately, creating that real-time feel users expect from modern messaging apps. Without WebSockets, achieving similarly instant delivery would require much less efficient techniques like constant polling.

Q282 How would you scale a chat application to support millions of concurrent users?

Scaling a chat app to millions of users typically involves distributing WebSocket connections across many servers, often using a connection gateway layer paired with a message broker like Kafka to route messages between servers. Since users connected to different servers still need to message each other, a shared pub-sub

Q283 What are the core requirements of a news feed system like the one on Instagram or Twitter?

A news feed system needs to collect posts from accounts a user follows and display them in a relevant, roughly chronological or ranked order whenever the user opens the app. Core requirements include efficiently storing posts, tracking follow relationships, and generating a personalized feed quickly, even for users following thousands of accounts. Additional considerations often include ranking algorithms and handling extremely popular accounts with millions of followers.

Q284 What is the fan-out on write approach for building a news feed?

Fan-out on write means that whenever a user posts something new, the system immediately pushes that post into the precomputed feed of every one of their followers. This makes reading a feed extremely fast later, since it is already prepared and waiting, but it can become expensive if an account has millions of followers to fan out to at once. It works especially well for users with a relatively normal, manageable number of followers.

Q285 What is the fan-out on read approach for building a news feed?

Fan-out on read means the system waits until a user actually opens their feed, then gathers and combines recent posts from everyone they follow at that moment. This avoids the expensive upfront cost of fanning out posts to potentially millions of followers immediately, but it can make loading the feed itself slower, especially for users following many accounts. This approach tends to work better for accounts with enormous numbers of followers.

Q286 How do real systems often combine both fan-out approaches?

Many real news feed systems use a hybrid approach, applying fan-out on write for typical users with a normal follower count, while using fan-out on read for extremely popular accounts with millions of followers. This avoids the worst case of instantly pushing a single post out to millions of feeds at once, while still keeping feed loading fast for the vast majority of ordinary users. This kind of hybrid design is a common real-world pattern for balancing competing performance tradeoffs.

Q287 Why is capacity estimation important in system design interviews?

Capacity estimation helps you reason about roughly how much traffic, storage, and processing power a system will actually need, which directly influences important design decisions like whether sharding or caching is necessary. It demonstrates that you can connect abstract design choices to real, concrete numbers rather than guessing. Even rough, approximate estimates are valuable for guiding sensible architectural tradeoffs.

Q288 How do you estimate storage requirements for a system?

To estimate storage requirements, you typically start with the expected number of records, such as users or posts, multiply that by the average size of each record, and account for growth over time. For example, one million users with roughly one kilobyte of profile data each would require about one gigabyte of storage just for profiles. Adding a buffer for growth and factoring in things like images or attachments helps create a more realistic estimate.

Q289 How do you estimate the number of servers needed for a system?

To estimate the number of servers needed, you typically calculate the expected number of requests per second, then divide by how many requests a single server can realistically handle based on typical benchmarks. For example, if a system expects ten thousand requests per second and one server can handle one thousand, you would need roughly ten servers, plus extra for redundancy. This rough estimate helps guide initial capacity planning, even though actual numbers require realworld testing to confirm.

Q290 What is a common mistake when doing back-of-the-envelope calculations?

A common mistake is spending too much time chasing perfectly precise numbers instead of focusing on the right order of magnitude, which is usually all that matters for early design decisions. Another mistake is forgetting to account for growth over time or peak traffic spikes, which can be far higher than average traffic. The goal of these estimates is to guide sensible, well-reasoned decisions

Q291 What is one of the most common mistakes candidates make in system design interviews?

One of the most common mistakes is jumping straight into drawing boxes and arrows before clarifying requirements, which often leads to solving the wrong problem entirely. Skipping this step can mean spending most of the interview designing for a scale or feature set the interviewer never actually asked for. Taking a few minutes upfront to ask clarifying questions almost always pays off later in the conversation.

Q292 Why is it a mistake to over-engineer a design too early?

Over-engineering too early, such as introducing sharding or microservices for a system that would comfortably run on a single server, can signal that a candidate does not know how to match a solution to actual requirements. It also wastes valuable interview time on complexity that was never needed in the first place. A stronger approach is to start simple and layer in complexity only once a clear, justified need for it appears.

Q293 Why is staying silent while designing considered a mistake?

Staying silent while designing is a mistake because interviewers are usually more interested in hearing your reasoning and tradeoffs than in seeing a perfect final diagram. Thinking out loud shows how you approach problems, handle uncertainty, and weigh different options against each other. A technically flawed design explained with clear reasoning often leaves a better impression than a silent, perfect-looking diagram.

Q294 What is latency, in one sentence?

Latency is the amount of time it takes for a single request to travel to a system and receive a response back, usually measured in milliseconds. Lower latency means a system feels faster and more responsive to the people using it. It is one of the most commonly discussed performance metrics in system design conversations.

Q295 What is throughput, in one sentence?

Throughput is the number of requests or operations a system can successfully process within a given period of time, such as requests per second. High throughput means a system can serve many users or tasks at once, which matters greatly for busy, high-traffic applications. It is often discussed alongside latency, since the two together describe a system's overall performance.

Q296 What is scalability, in one sentence?

Scalability is a system's ability to handle increasing amounts of traffic or data by adding more resources, whether through more powerful machines or more machines altogether. A highly scalable system can grow smoothly from a handful of users to millions without needing to be redesigned from scratch. It remains one of the central goals behind almost every decision made in system design.

Q297 How should you approach a system design interview question?

You should start by clarifying requirements and constraints, asking questions about scale, features, and priorities before jumping into any specific design. From there, sketch a high-level design covering the main components, then progressively go deeper into specific areas like data storage, scaling, and reliability as time allows. Communicating your reasoning clearly throughout the process is often just as important as the final design itself.

Q298 Why is it important to discuss tradeoffs in a system design interview?

Discussing tradeoffs shows that you understand there is rarely one single correct answer in system design, and that every decision involves balancing competing priorities like cost, complexity, and performance. Interviewers are often more interested in your reasoning process than in whether you land on their exact preferred solution. Explicitly calling out tradeoffs demonstrates a deeper, more mature understanding of real-world engineering decisions.

Q299 What is a good habit to build when learning system design?

A good habit is to regularly study how real, well-known systems are actually built, since seeing how companies solve problems at massive scale reinforces the concepts in a memorable, practical way. Practicing by designing simplified versions of popular applications, like a URL shortener or chat app, also helps translate theory into hands-on skill. Consistent, gradual practice over time is far more effective than trying to memorize everything at once.

Q300 What is the key takeaway for continuing your system design journey beyond the basics?

The key takeaway is that system design is a deep, ever-growing field, and the concepts covered here are the essential foundation on which everything more advanced is built. Every large-scale system, no matter how complex it looks, is ultimately built from combinations of the fundamental building blocks covered throughout this guide. Keep practicing, keep reading about real systems, and gradually these advanced topics will feel just as intuitive as the basics once did.